



CIRCUITOS DIGITALES Y MICROCONTROLADORES 2026

Facultad de Ingeniería
UNLP

Programación de Microcontroladores
con Lenguaje C

Ing. José Juárez

Empecemos repasando los tipos de datos en C

• Tipo		#bits	Rango de representación	bytes
signed char		8bits	-128 a +127	1
unsigned char		8bits	0 a 255	1
short int		16bits	-32768 a 32767	2
unsigned short		16bits	0 a 65535	2
int		???	(depende del compilador)	
long		32bits	- 2147483648 a 2147483647	4
unsigned long		32bits	0 a 4294967295	4
float	(IEEE32)	32bits	1.17549x10-38 a 3.40282x10+38	4
double	(IEEE64)	64bits	2.22507x10-308 a 1.79769x10+308	8

- Notar que la cantidad de bits asignada a un `int` puede variar de compilador en compilador manteniéndose la relación:

`sizeof(char) < sizeof(short) <= sizeof(int) <= sizeof(long)`

Tipos de datos en C

- **Primer lección** : Utilizar el tipo de dato óptimo para la arquitectura del uC
- **Arquitecturas de 8 bits:**
 - el tipo de dato más adecuado es **signed char** o **unsigned char**
 - Operaciones en ALU de 8bits en un ciclo de reloj
 - Operaciones con datos enteros de mayor precisión (16 o 32 bits) deben realizarse por código (generalmente incluido como funciones de biblioteca del compilador)
 - El uso de datos **float** o **double** en arquitecturas con ALU de punto fijo solo debe hacerse cuando es estrictamente necesario. Las operaciones con este tipo de dato se realizan mediante rutinas de biblioteca y consumen muchos recurso del uC (ciclos de reloj y bytes de memoria).
- **Arquitecturas de 32 bits:**
 - el tipo de dato más adecuado es **long int** o **unsigned long int**
- **Segunda lección** : Utilizar el signo adecuado.
 - **Si no importa el signo utilizar unsigned.**
 - Los modificadores **unsigned** / **signed** indican al compilador que instrucciones de assembler debe utilizar.
 - Muchas arquitecturas son más eficientes cuando operan con números sin signo.
 - Si la variable es **signed** se utilizarán instrucciones que operan sobre **Ca2**, caso contrario no.

Alcance de las variables en C

- **Locales:**

- Son variables que viven dentro del ámbito de una función { } cuando está se está ejecutando
- Desde el punto de vista del almacenamiento en memoria, las variables locales se alojan en los registros CPU o en la memoria de pila (stack)
- Por esta razón, solo la función donde están declaradas conoce su ubicación, haciéndolas invisible a otras funciones.
- Una vez finalizada la función son descartadas liberando memoria.
- Un beneficio adicional de las variables locales es que permite modularidad y por tanto la reutilización de código

- **Globales:**

- Son variables que pueden ser accedidas por desde cualquier parte del programa.
- Se declaran fuera de toda función y son alojadas en direcciones fijas de memoria RAM (puede requerir modos de direccionamiento extendido oara especificar la dirección de memoria)
- Cualquier función puede acceder y modificar su contenido, incluso en un servicio de interrupción
- Las variables globales se inicializan durante la ejecución del código de "startup"

Modificadores y tipo de almacenamiento

- **Static:**

- Una variable local declarada como static posee una dirección fija de memoria RAM que se conserva durante toda la ejecución del programa. De esta manera su valor se mantiene entre las sucesivas llamadas a la función.
- Una variable global declarada como static se convierte en una variable “privada” y su alcance se limita solo al archivo .c donde está definida.
- Una función declarada como static se convierte en una función “privada” y su alcance se limita solo al archivo .c donde está definida.

- **Volatile:**

- Advierte al compilador que la variable puede cambiar su valor por acción externa al programa y NO debe aplicar optimizaciones sobre dicha variable
- Ejemplo 1: todas las variables que representan los registros del MCU deben ser volátiles
- Ejemplo 2: variables globales modificadas por una interrupción deben ser volátiles

- **Const:**

- Indica al compilador que la variable es READ_ONLY y por lo tanto no puede modificarse en otra parte del programa (evita errores de escritura indeseados).
- El compilador puede almacenar este tipo de variables en ROM, optimizando el uso de la memoria RAM (uso de PROGMEM).

Constantes en C

- Constantes

X+3;	←	Cte decimal 3
'F'	←	Caracter ASCII
"HOLA"	←	Cadena de caracteres ASCII

- Prefijos

0	←	octal
0b	←	binario
0x	←	hexadecimal

- Sufijos

U, L, UL, F

Ejemplo: **#define** CLK 16000000**UL**

Enumeraciones

Las Enumeraciones son un conjunto de constantes enteras que limitan el valor que una variable de este tipo puede tomar.

```
enum tag_name {  
    NOMBRE_0 [=CONST_0],  
    NOMBRE_1 [=CONST_1],  
    ...  
    NOMBRE_n [=CONST_n]  
};
```

- Ejemplo:

```
enum meses { ENE=1, FEB, MAR, ABR, MAY, JUN, JUL, AGO, SEP, OCT, NOV, DIC };
```

```
meses var1, var2 ;       Declaro variables del tipo meses
```

```
var1 = FEB;
```

Operadores Lógicos de bits en C

		AND	OR	EX-OR	Inverter
A	B	A&B	A B	A^B	Y=~B
0	0	0	0	0	1
0	1	0	1	1	0
1	0	0	1	1	
1	1	1	1	0	

- Ejemplos:

```
unsigned char a=0x35, b=0x0F, c ;
```

```
c=a & b;
```

```
c=a | b;
```

```
c= a ^ b;
```

```
c= ~ a;
```

No confundir con los operadores de comparación: &&, ||, !=, !, <=, etc

Arreglos: ejemplo multidimensional

```
#define TRUE 1
```

```
#define FALSE 0
```

```
char getkey (char * row , char * col);
```

← Prototipo de función

```
const char keys[4][3] = { '1', '2', '3',  
                          '4', '5', '6',  
                          '7', '8', '9',  
                          '*', '0', '#' };
```

← Definición de una matriz que representa el teclado

```
void main(void) {  
    char press, row , col;  
    while(1) {  
        if( ( press= getkey( &row , &col ) ) == TRUE);  
            putchar (keys[row][col]);  
    }  
}
```

← si se presionó una tecla la función getkey
modificará el valor de row y col para indicar cuál es
dicha tecla.